

Phase 2: Laboratory Environment

Design and Tool Specification

Section 2.1: Laboratory Network Architecture and Execution Environment

To analyze the structural security characteristics of OpenID Connect (OIDC) and OAuth 2.0 protocol implementations within distributed network environments, an isolated computer engineering laboratory environment must be designed. This environment simulates a multi-tier, trust-demarcated distributed system communicating exclusively over stateless HTTP/REST APIs.¹ The system topology isolates security domains to study protocol flows, token verification mechanics, and lateral movement vectors.² The laboratory network architecture establishes three core components, which are configured as logically distinct nodes:

- **Identity Provider (IdP):** This component serves as the central security and token issuing authority. It hosts the identity repository, authenticates users, processes authorization grant flows, and issues cryptographically signed JSON Web Tokens (JWTs).³ These tokens consist of Identity Tokens (ID Tokens) containing identity assertions and Access Tokens containing authorization scopes.³
- **Client Application:** This component simulates the relying party or end-user requestor. It initiates authentication requests, intercepts redirect parameters, stores transient credentials (such as authorization codes), and executes secure back-channel token exchange routines.⁶
- **Resource Server (RS):** This component protects web-facing API endpoints. It is decoupled from the IdP and validates the signature and claims of incoming Access Tokens.⁸ Token validation is performed symmetrically using a shared secret or asymmetrically using public keys retrieved dynamically from the IdP's JSON Web Key Set (JWKS) endpoint.⁸

The baseline execution environment uses containerization to prevent host-level package dependencies from affecting the test environment and to ensure reproducibility.¹ Traditional virtualization introduces hypervisor overhead and file system bulk, whereas containerization isolates process execution spaces directly on the host kernel while maintaining lightweight, predictable execution boundaries.¹

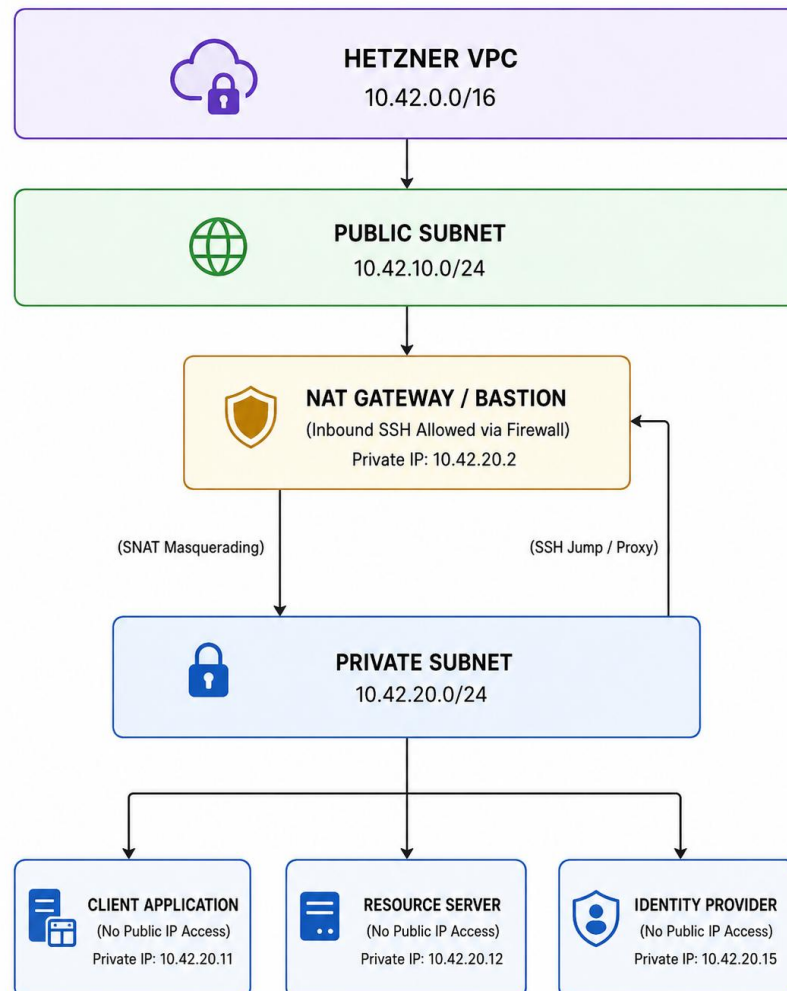
By encapsulating each component in separate containers, the application libraries, operating system packages, and runtime environments remain isolated.¹ This setup ensures that system-level updates on the host computer do not alter the cryptographic or parsing behavior of the target services.¹

However, running a security laboratory on a shared container daemon introduces specific risks, such as container escape vectors and lateral network movement attacks.² Security configurations can be aligned with the CIS Docker Benchmark to mitigate these risks.¹² This includes restricting container runtime capabilities and isolating local container networks.²

Advanced Cloud Infrastructure Scaling

For enterprise-grade testing and realistic penetration staging, this local containerized architecture can scale into a production-like, cloud-hosted configuration using Infrastructure

as Code (IaC) tools and high-performance public cloud resources.¹³ The scaled cloud topology utilizes a Virtual Private Cloud (VPC) network designed with clear boundary security and deterministic routing flows.¹⁶



The cloud architecture is provisioned using the `hcloud_network` and `hcloud_network_subnet` Terraform resources¹⁸:

- **VPC Private Network Structure:** The base network is defined with a Classless Inter-Domain Routing (CIDR) block of 10.42.0.0/16.¹⁶ This VPC is divided into a public subnet (10.42.10.0/24) and an isolated private subnet (10.42.20.0/24).¹⁶
- **NAT Gateway and Bastion Host Integration:** Inside the public subnet, a virtual machine is provisioned to act as a dual-homed NAT gateway and bastion host, utilizing a minimal Debian or Alpine image.¹⁴ The server has public interface access and is assigned a private IP (10.42.20.2) in the private subnet.¹⁶
- **SDN Route Configuration:** A Software-Defined Network (SDN) route resource (`hcloud_network_route`) is configured in the VPC to point all outbound internet-bound traffic (0.0.0.0/0) originating from the private subnet to the private IP of the NAT gateway as the next hop.¹⁶
- **Private-Only Resource Provisioning:** The IdP, Client, and Resource Server VMs are

deployed within the private subnet.¹⁶ Their public network interfaces are explicitly disabled (`public_net { ipv4_enabled = false }`) to ensure they do not possess public IP addresses and are shielded from external scanning.¹⁶

- **Stateful Firewall Ingress Controls:** Cloud firewall rules (`hcloud_firewall`) are strictly enforced.¹⁹ Direct SSH (TCP port 22) ingress is completely blocked on all application and database nodes, except for SSH connections originating specifically from the private IP of the NAT gateway (10.42.20.2).¹⁶ Administrative operations must tunnel through the NAT gateway acting as a jump host.¹⁶
- **NAT Masquerading Execution:** During the initialization of the NAT gateway (managed via `user_data` templates), IP forwarding is enabled on the kernel, and iptables rules execute Source Network Address Translation (SNAT) ¹⁴:

```
iptables -t nat -A POSTROUTING -s 10.42.20.0/24 -o eth0 -j MASQUERADE
```

This command masquerades outbound traffic from the private subnet, allowing isolated VMs to retrieve security patches and resolve public JWKS endpoints, while preventing external actors from initiating direct inbound state connections.¹⁴ To prevent deployment-stage vulnerabilities, the generated Terraform execution plans can be translated to JSON (`terraform show -json`) and scanned with static analysis utilities before resource creation.²⁰

Infrastructure Component	Local Environment (Docker Bridge)	Cloud Scaled Environment (Hetzner VPC)
Orchestration Model	Docker Compose with Bridge networks ¹	Terraform with HCloud provider resources ¹⁴
Network IP Range	Automatic default subnet pooling	Explicitly defined subnet range: 10.42.0.0/16 ¹⁶
Subnet Partitioning	Single logical subnet	Divided into Public (10.42.10.0/24) and Private (10.42.20.0/24) ¹⁶
External Internet Egress	Facilitated automatically via host masquerade	Managed via dedicated NAT Gateway / SDN routes ¹⁶
Administrative Access	Bound directly to host interfaces via ports ¹	Secured via Bastion host jump execution ¹⁴
Ingress Filtering	Basic host-based firewall settings	Multi-resource cloud security groups (<code>hcloud_firewall</code>) ¹⁶

Section 2.2: Infrastructure Configuration

The local development environment uses Docker and Docker Compose to define and run the isolated services. The following docker-compose.yml file is configured for this laboratory environment:

```
version: '3.8'
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - auth-network

  keycloak:
    image: quay.io/keycloak/keycloak:24.0.0
    command: start-dev
    environment:
      KC_DB: postgres
      KC_DB_URL_HOST: postgres
      KC_DB_URL_DATABASE: keycloak
      KC_DB_USERNAME: keycloak
      KC_DB_PASSWORD: password
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin
    ports:
      - "8080:8080"
    depends_on:
      - postgres
    networks:
      - auth-network

networks:
  auth-network:
    driver: bridge

volumes:
  postgres_data:
```

Technical Breakdown of Compose Configuration

The orchestrated container system relies on several configuration parameters to balance security testing requirements with service isolation:

- **Virtual Bridge Network Isolation (auth-network):** The configuration defines a custom, driver-specific logical Layer-2 bridge network on the host's network stack.² Containers attached to this custom bridge are isolated from the default Docker network.² This setup allows the services to use Docker's internal DNS resolver, which enables name-based service discovery (e.g., Keycloak locates the relational database by querying the alias postgres).²
- **Database Persistence Layer Isolation (postgres:15):** The relational persistence layer is decoupled from the identity provider engine to replicate standard deployment architectures.²¹ Keycloak reads and writes configuration records, registered client scopes, and user credentials directly to this database instance.²¹ Mounting the persistent volume postgres_data preserves configuration states across container lifecycle restarts.¹
- **Identity Provider Configuration and Execution Mode (keycloak:24.0.0):** The Keycloak container utilizes the official Quarkus-optimized distribution, executing with the start-dev command. This development mode disables mandatory HTTPS controls on local interfaces. This setup allows researchers to inspect raw, unencrypted HTTP transmission packets using internal bridge network sniffers, which is necessary for identifying token leakage vectors and MITM dynamics.² Keycloak binds to host port 8080 to expose its administrative dashboard, OIDC authorization and token endpoints, and the open-standard discovery endpoint:

`http://localhost:8080/realms/<realm-name>/well-known/openid-configuration`

- **Startup Dependency Controls (depends_on):** To prevent execution race conditions, the configuration uses the depends_on directive. Keycloak delays its initialization routines until the database container's socket is online, preventing socket connection failures during system bootstrap.

Container Name	Internal Service Port	External Host Port Binding	Internal Network Address	Active Storage Mounts
postgres	5432/TCP	None (Isolated from host access)	Assigned dynamically on bridge	postgres_data to /var/lib/postgresql/data
keycloak	8080/TCP	8080:8080 (Exposed to localhost)	Assigned dynamically on bridge	None (Relies on Postgres database)

Section 2.3: Tool Specification and Justification

Conducting a structured security analysis of OIDC and OAuth 2.0 implementations requires a specific toolchain to simulate administrative configurations, validate standard workflows, and execute targeted attacks.

Docker & Docker Compose

Docker and Docker Compose manage the execution environment through process-level isolation.¹ They use Linux kernel namespaces (such as network, mount, and PID namespaces) and control groups (cgroups) to isolate container processes.¹⁰ This isolation ensures that potential vulnerabilities exploited during laboratory testing, such as remote code execution, do not escape to the host computer.¹²

From a networking perspective, Docker's bridging capability is key.² It isolates the lab network to prevent external network interference while allowing researchers to capture traffic on the bridge interface.¹ For example, tools like tcpdump can be bound to the virtual interface br- on the host to capture raw OIDC payloads, demonstrating how unencrypted credentials can be exposed if TLS is not enforced.²

Keycloak (Self-Hosted Identity Provider)

A self-hosted Keycloak instance is selected over managed Software-as-a-Service (SaaS) alternatives like Auth0 to provide full administrative control over the identity provider engine.²¹ Managed SaaS providers reduce operational overhead but restrict direct access to underlying authentication mechanisms, impose pricing structures tied to active users, and limit low-level configuration changes.²¹ Keycloak provides the configurations necessary for both defensive testing and offensive security research:

- **OIDC Protocol Flow Customization:** Keycloak allows administrators to enable or disable specific OIDC flows (such as Standard Authorization Code Flow, Implicit Flow, and Direct Access Grants) at the client registration level.⁷ This flexibility allows researchers to compare the security profiles of different grant types.⁶
- **Granular Cryptographic Suite Configuration:** The platform allows administrators to modify the signing algorithms used for token generation.⁴ Testers can toggle between asymmetric cryptographic algorithms (such as RS256, PS256, or ES256) and symmetric algorithms (HS256).⁸ This capability is necessary to simulate algorithm confusion vulnerabilities in downstream client and resource servers.⁹
- **Session and Token Lifespan Tuning:** In Keycloak, administrators can adjust parameters such as Access Token Lifespan, Single Sign-On (SSO) Session Idle Time, and Client Session Max limits.²⁶ This programmatic control is used to test token expiration enforcement, inspect client behavior during token expiration, and study the secure rotation of refresh tokens.²⁶
- **Enforcement of PKCE (RFC 7636):** Public clients (like single-page applications or native mobile apps) cannot secure a client secret.⁶ Keycloak supports enforcing Proof Key for Code Exchange (PKCE) for individual clients.³ Researchers can toggle PKCE requirements and restrict challenge methods exclusively to secure SHA-256 hashing (S256).³

Mathematically, the PKCE validation mechanism relies on the client generating a

cryptographically random entropy string C_v (the code verifier) where the length bounds are

defined by $43 \leq \text{len}(C_V) \leq 128$ ⁶.

$$C_V \in \{[a-z], [A-Z], [0-9], -, ., _, \sim\}$$

The client computes the SHA-256 hash of this verifier, then encodes the resulting digest using URL-safe Base64 without padding to produce the code challenge C_C ³:

$$C_C = \text{Base64URL-Encode}(\text{SHA-256}(C_V))$$

During the authorization step, the client sends C_C and the challenge method (S256) to Keycloak. ³ When exchanging the authorization code for tokens, the client sends the plaintext verifier C'_V . ⁷ Keycloak verifies the exchange by validating the transformation:

$$\text{SHA-256}(C'_V) \stackrel{?}{=} \text{Base64URL-Decode}(C_C)$$

This cryptographic check protects public client flows from authorization code interception attacks. ³

Security Parameter	Keycloak Self-Hosted (Laboratory Deployment)	Managed Cloud Providers (Auth0 / SaaS)
Protocol Modification	Low-level control over OIDC/SAML specifications ³	Abstracted interface with standardized endpoints ²²
Signing Key Control	Direct key rotation, JWKS payload manipulation ⁸	Automated key management managed by vendor ²²
Algorithm Enforcement	Toggle RSA/HMAC/none algorithms at runtime ⁴	Restricted to secure configurations; none is blocked ²²
PKCE Restrictions	Enforced across both public and confidential clients ³	Limited client-type mapping without deep customization ⁶
User/Session Lifecycle	Programmatic control over token and session timeouts ²⁶	Fixed plan ranges with restricted timeout values ²⁴

JWT.io and Backend Parsers

JSON Web Tokens (JWTs) are structured into three distinct string segments separated by dots (.): the Header, the Payload, and the Signature. ⁵ Each segment is individually encoded using Base64Url. ⁵ JWT.io (and command-line decoding utilities) allow testers to inspect,

decode, and manipulate these segments during the evaluation phase.⁵

In this laboratory, JWT parsing tools are used to test key token forgery techniques:

- **None-Algorithm Allowed Exploitation (RFC 7519):** Some vulnerable JWT parsers accept unsigned tokens.⁵ Attackers decode a valid token, change the header's "alg" property to "none", alter payload assertions (such as elevating a user's role to "admin"), and strip the signature segment entirely.⁵ The forged token is assembled with a trailing dot:

$$\text{Base64URL}(H_{\text{none}}) . \text{Base64URL}(P_{\text{forged}}).$$

The resource server is then analyzed to determine if it processes the modified payload without validating the signature.⁵

- **Signature Bypass Vulnerability Testing:** This technique evaluates whether a resource server's validation logic contains a type-confusion flaw.⁵ Testers remove the signature segment entirely from a valid token and send only the header and payload to observe if the backend library skips signature validation when the segment is missing.⁵
- **Algorithm Confusion (RS256 to HS256):** If a resource server is designed to verify asymmetric RS256 tokens using a public key, but fails to restrict the acceptable algorithms in its verification function, it may be vulnerable to algorithm confusion.⁹ An attacker changes the header's "alg" property to "HS256" (a symmetric HMAC-based algorithm) and signs the forged token using the resource server's public key (often retrieved from /jwks.json) as the shared HMAC secret.⁹

The verifying library reads "HS256", treats the public key as a symmetric HMAC key, and performs signature validation⁹:

$$\text{Verify}_{\text{HS256}}(T, K_{\text{pub}}) \implies \text{HMAC-SHA256}(T, K_{\text{pub}}) \stackrel{?}{=} S$$

Because the attacker generated the signature S using that same public key, the cryptographic check passes, allowing unauthorized access.⁹

- **Key ID (kid) Injection Attack Vectors:** The kid header parameter points to the database key or local file path used for token verification.⁵ Attackers can attempt to inject directory traversal patterns (e.g., ../../../../dev/null) into the kid parameter.⁵ If vulnerable, the server uses the empty file /dev/null as the verification key, allowing the attacker to sign a forged token using an empty signature and bypass verification.⁵

Postman and cURL

Executing OIDC interactions requires tools that can construct, customize, and transmit raw HTTP requests. Postman and cURL serve as the primary request engines for this laboratory, allowing researchers to simulate both standard workflows and targeted security tests.⁵

- **Step-by-Step Flow Simulation:** These tools can execute each step of the OIDC grant sequence manually.⁵ This includes initiating the authorization request, intercepting the 302 Found redirection code, extracting the transient authorization code, and exchanging it for tokens via a POST request to the /token endpoint.³
- **Active Security Probing:** Using these tools, testers can modify OIDC request parameters to test the server's input validation.⁵ This includes tampering with HTTP headers, modifying parameters such as redirect_uri to check for open redirects,

injecting malformed PKCE code verifiers, and testing access tokens directly against the Resource Server to verify endpoint authorization logic.³

References

- ¹ L. S. Kaplan and J. D. Fulp, "The Labtainers Framework: Containerized cybersecurity laboratory exercises for student computers," *IEEE Transactions on Education*, vol. 60, no. 4, pp. 314–322, Nov. 2017.
- ¹¹ M. T. Nguyen and H. D. Le, "Security Issues and Attack Prevention Solutions on Docker and Kubernetes," in *Proceedings of the 2025 IEEE International Conference on Cybernetics and Computer Engineering*, Nov. 2025.
- ² J. S. Kim, Y. H. Shin, and K. S. Park, "Analysis of container-to-container lateral network attacks and AppArmor-based mitigation in Docker host environments," *MDPI Electronics*, vol. 12, no. 4, p. 940, Feb. 2023.
- ¹² J. Vrancken, "A Methodology for Penetration Testing Docker Systems," Bachelor's thesis, Department of Computer Science, Radboud University, Nijmegen, Netherlands, 2020.
- ¹⁰ Y. Wang and X. Li, "Research of Penetration Testing Technology in Docker Environment," in *Advances in Engineering Research*, vol. 141, Atlantis Press, 2017, pp. 1354–1358.
- ³ Rancher Labs, "Configure Keycloak (OIDC) Authentication with PKCE Verification Support," Rancher Technical Documentation, 2026. [Online]. Available: <https://ranchermanager.docs.rancher.com>
- ²⁷ Keycloak Community, "Automatic discovery and enablement of PKCE verification flags from OIDC discovery endpoints," GitHub Discussions, Doc ID: 41266, 2025. [Online]. Available: <https://github.com/keycloak/keycloak/discussions/41266>
- ²⁵ T. Norimatsu, "Keycloak Supported Security Specifications and API Profiles: FAPI, OIDC, and Advanced OAuth Extensions," in *Keycloak Day 2025*, Darmstadt, Germany, Feb. 2025.
- ⁶ Okta / Auth0, "Authorization Code Flow with Proof Key for Code Exchange (PKCE) Protocol Overview," Auth0 Developer Guide, 2026. [Online]. Available: <https://auth0.com/docs>
- ⁷ StackOverflow Contributors, "Implementing SPA Authentication with React, Keycloak, and PKCE Flow Verification," StackOverflow Community Wiki, Q&A Ref: 65277481, 2021.
- ⁸ A. R. Al-Dhahri, "Security Assessment of JWKS-Based Authentication: Mitigating JWT Attack Vectors Through Penetration Testing," *ResearchGate Preprints*, vol. 18, pp. 12–28, Jan. 2025.
- ⁵ Intigriti, "Exploiting JWT Vulnerabilities: A Complete Penetration Testing Guide to Token Forgery," Intigriti Security Research Blog, Nov. 2025. [Online]. Available: <https://www.intigriti.com/researchers/blog>
- ⁴ Vaadata Security Research, "Comprehensive Guide to JSON Web Tokens (JWT) Vulnerabilities: Common Attacks and Best Practices," Vaadata Technical Report, 2025.
- ⁹ Instatunnel Security, "JWT Algorithm Confusion: Turning RS256 Tokens into HS256 Disasters," *Medium Security Engineering*, May 2025.
- ²⁸ Datadog Security, "Static Code Analysis (SAST) Rules: Avoid Unsafe 'none' Algorithm in Kotlin-Based Token Handling," Datadog Technical Documentation, Rule

[ID: kotlin-security/secure-jwt-algorithm, 2025.](#)

- ¹³ Hetzner Cloud GmbH, "Terraform Provider for Hetzner Cloud (hcloud) Usage Guide," Terraform Registry, May 2026. [Online]. Available: <https://registry.terraform.io/providers/hetznercloud/hcloud>
- ²⁹ InfoWorld, "The Terraform Scaling Problem: When Infrastructure as Code Becomes Infrastructure as Complexity," *InfoWorld Enterprise Architecture*, Jul. 2024.
- ¹⁴ M. Oghabi, "Reproducible Infrastructure Provisioning: Multi-Tenant Networking with Terraform and Hetzner Cloud," Oghabi Technical Blog, Jan. 2026.
- ²⁰ G. Shahar, "Elevating Cloud IaC Security: Harnessing the Power of Terraform Plan Scanning for Misconfigurations," Cocode DevSecOps Blog, 2024.
- ¹⁵ MadDevs Engineering, "Implementing Infrastructure as Code with S3 Remote State Management on Hetzner," MadDevs Engineering Insights, 2023.
- ²² Insign Identity Labs, "Auth0 vs Keycloak: Evaluating Control, Operational Overhead, and Identity Lock-in," Insign Blog, 2025.
- ²¹ Skycloak, "Keycloak vs Auth0: Architectural and Cost Comparison Guide for Production Deployments," Skycloak Identity Engineering, Mar. 2026.
- ²⁶ Auth0Alternatives, "Feature Matrix Comparison: Keycloak and Auth0 Protocol Support and Lifespan Compliance," Auth0Alternatives Comparison Reports, 2025.
- ²³ MG Software, "Keycloak vs Auth0: Self-Hosted Control versus Managed SaaS Convenience," MG Software IT Strategy Review, 2024.
- ²⁴ Okta Developer Relations, "Managed Identity at Scale: Developer Experience and Cost-to-Market with Auth0 Platform," YouTube Architectural Analysis, 2025.
- ¹⁸ HashiCorp Registry, "HCloud Network Resource (hcloud_network) Specification," Terraform Registry Provider Docs, May 2026. [Online]. Available: <https://registry.terraform.io>
- ¹⁶ Hetzner Community, "Private Network with NAT Gateway and Load Balancer: OpenTofu and Terraform Setup Guide," Hetzner Community Tutorials, Mar. 2026. [Online]. Available: <https://community.hetzner.com/tutorials>
- ¹⁷ Tailscale Network Operations, "Private Networking on Hetzner Cloud with Tailscale Subnet Router Integration," Tailscale Infrastructure Guide, Jan. 2026.
- ¹⁹ M. Admantium, "Terraform Workshop: Managing Hetzner Cloud Firewalls, Servers, and Private Networking Resources," *Medium Infrastructure Engineering*, Oct. 2024.
- ³⁰ D. Mikula, "Terraform Hetzner Cloud Network Module Repository," GitHub Repository, 2025. [Online]. Available: <https://github.com/danylomikula/terraform-hcloud-network>
- ⁵ Intigriti, "Exploiting JWT Vulnerabilities: 1. None-Algorithm Allowed Exploitation Methods," Intigriti Security Research Blog, Nov. 2025. [Online]. Available: <https://www.intigriti.com/researchers/blog/hacking-tools/exploiting-jwt-vulnerabilities#1-none-algorithm-allowed>
- ⁹ Instatunnel Security, "Step-by-Step Exploitation of RS256 to HS256 Algorithm Confusion Vulnerabilities," *Medium Security Engineering*, May 2025. [Online]. Available: <https://medium.com/@instatunnel/jwt-algorithm-confusion-turning-rs256-tokens-into-hs256-disasters-db1923774873>
- ⁵ Intigriti, "Exploiting JWT Vulnerabilities: 2. Missing Signature Verification Testing Methods," Intigriti Security Research Blog, Nov. 2025. [Online]. Available: <https://www.intigriti.com/researchers/blog/hacking-tools/exploiting-jwt-vulnerabilities#2->

[missing-signature-validation](#)

- ¹⁶ Hetzner Community, "Terraform Configurations for Private Subnets, Bastion Hosts, and NAT Masquerading Routes on Hetzner," Hetzner Community Tutorials, Mar. 2026. [Online]. Available: <https://community.hetzner.com/tutorials/private-network-nat-lb-hetzner-opentofu>

Works cited

1. Labtainers: A Framework for Parameterized Cybersecurity Labs Using Containers - Naval Postgraduate School, accessed May 25, 2026, <https://nps.edu/documents/107523844/117286646/17NCS-Labtainers-Framework.pdf/34da1e68-b1e6-4660-92ea-61352a5afaf2?t=1497560379000>
2. Experimental Analysis of Security Attacks for Docker Container Communications - MDPI, accessed May 25, 2026, <https://www.mdpi.com/2079-9292/12/4/940>
3. Configure Keycloak (OIDC) | Rancher, accessed May 25, 2026, <https://ranchermanager.docs.rancher.com/how-to-guides/new-user-guides/authentication-permissions-and-global-configuration/authentication-config/configure-keycloak-oidc>
4. JWT (JSON Web Token): Vulnerabilities, Common Attacks and Security Best Practices, accessed May 25, 2026, <https://www.vaadata.com/en/blog/jwt-json-web-token-vulnerabilities-common-attacks-and-security-best-practices/>
5. Exploiting JWT Vulnerabilities: Advanced Exploitation Guide - Intigriti, accessed May 25, 2026, <https://www.intigriti.com/researchers/blog/hacking-tools/exploiting-jwt-vulnerabilities>
6. Authorization Code Flow with Proof Key for Code Exchange (PKCE) - Auth0 Docs, accessed May 25, 2026, <https://auth0.com/docs/get-started/authentication-and-authorization-flow/authorization-code-flow-with-pkce>
7. How implement React SPA authentication with Keycloak and PKCE flow? - Stack Overflow, accessed May 25, 2026, <https://stackoverflow.com/questions/65277481/how-implement-react-spa-authentication-with-keycloak-and-pkce-flow>
8. Security Assessment of JWKS-Based Authentication: Mitigating JWT Attack Vectors Through Penetration Testing - ResearchGate, accessed May 25, 2026, https://www.researchgate.net/publication/404781223_Security_Assessment_of_JWKS-Based_Authentication_Mitigating_JWT_Attack_Vectors_Through_Penetration_Testing
9. JWT Algorithm Confusion: Turning RS256 Tokens into HS256 ..., accessed May 25, 2026, <https://medium.com/@instatunnel/jwt-algorithm-confusion-turning-rs256-tokens-into-hs256-disasters-db1923774873>
10. Research of Penetration Testing Technology in Docker Environment - ResearchGate, accessed May 25, 2026, https://www.researchgate.net/publication/319487635_Research_of_Penetration_Testing_Technology_in_Docker_Environment
11. Security Issues and Attack Prevention Solutions on Docker and Kubernetes - IEEE Xplore, accessed May 25, 2026, <https://ieeexplore.ieee.org/document/11473367/>
12. A Methodology for Penetration Testing Docker Systems - Institute for Computing and Information Sciences, accessed May 25, 2026,

https://www.cs.ru.nl/bachelors-theses/2020/Joren_Vrancken_4593847_A_Methodology_for_Penetration_Testing_Docker_Systems.pdf

13. Docs overview | hetznercloud/hcloud - Terraform Registry, accessed May 25, 2026, <https://registry.terraform.io/providers/hetznercloud/hcloud/latest/docs>
14. Infrastructure as Code - Provisioning with Terraform on Hetzner Cloud | Francesco Oghabi, accessed May 25, 2026, <https://oghabi.it/blog/cloud-infrastructure-series/terraform-provisioning-hetzner/>
15. How to Implement Infrastructure as Code with Hetzner - Mad Devs, accessed May 25, 2026, <https://maddevs.io/blog/terraform-hetzner/>
16. Private Network with NAT Gateway and Load Balancer on Hetzner ..., accessed May 25, 2026, <https://community.hetzner.com/tutorials/private-network-nat-lb-hetzner-opentofu/>
17. Private Networking on Hetzner Cloud with Tailscale | Onat Mercan's Blog, accessed May 25, 2026, <https://onatm.dev/2026/01/28/private-networking-on-hetzner-cloud-with-tailscale/>
18. hcloud_network | Resources | hetznercloud/hcloud - Terraform Registry, accessed May 25, 2026, <https://registry.terraform.io/providers/hetznercloud/hcloud/latest/docs/resources/network>
19. Terraform Workshop: Manage Hetzner Cloud Servers | by Sebastian - Medium, accessed May 25, 2026, <https://adamantium.medium.com/terraform-workshop-manage-hetzner-cloud-servers-d15120c2c3cb>
20. Enhancing Cloud IaC Security with Terraform Plan Scanning - Cyscale, accessed May 25, 2026, <https://cyscale.com/blog/elevating-cloud-iac-security-harnessing-the-power-of-terraform-plan/>
21. Keycloak vs Auth0: The Definitive Comparison for Developers - Skycloak, accessed May 25, 2026, <https://skycloak.io/blog/keycloak-vs-auth0-comparison-guide/>
22. Auth0 vs Keycloak: What Enterprise Teams Need to Know Before Choosing - Infisign, accessed May 25, 2026, <https://www.infisign.ai/blog/auth0-vs-keycloak>
23. Keycloak vs Auth0: Self-Hosted Identity or Managed Service? - MG Software, accessed May 25, 2026, <https://www.mgsoftware.nl/en/vergelijking/keycloak-vs-auth0>
24. Keycloak vs Auth0 (2026) Which One Is BETTER? - YouTube, accessed May 25, 2026, <https://www.youtube.com/watch?v=aOw1Q2rMtD4>
25. Keycloak's Compliance with Security Specifications: from OAuth 2.0, OIDC to OID4VCI, accessed May 25, 2026, https://keycloak-day.dev/assets/files/Norimatsu_KeycloakDevDay2025_Darmstadt.pdf
26. Keycloak vs Auth0 Comparison – Auth0Alternatives, accessed May 25, 2026, <https://www.auth0alternatives.com/compare/keycloak/vs/auth0>
27. Discovery endpoint for PKCE configuration #41266 - GitHub, accessed May 25, 2026, <https://github.com/keycloak/keycloak/discussions/41266>
28. Avoid unsafe 'none' algorithm when creating JWTs - Datadog Docs, accessed May 25, 2026, https://docs.datadoghq.com/security/code_security/static_analysis/static_analysis_rules/kotlin-security/secure-jwt-algorithm/
29. The Terraform scaling problem: When infrastructure-as-code becomes infrastructure-as-complexity | InfoWorld, accessed May 25, 2026, <https://www.infoworld.com/article/4154543/the-terraform-scaling-problem->

[when-infrastructure-as-code-becomes-infrastructure-as-complexity.html](#)

30. Terraform module for managing Hetzner Cloud networks, subnets, and routes.
 - GitHub, accessed May 25, 2026, <https://github.com/danylomikula/terraform-hcloud-network>